

**Lebanese American University**  
**School of Arts and Sciences**  
**Department of Computer Science and Mathematics**



A database design for the **Esports Team Performance Tracker**

*By:*

**Kareem Naous**

202407991

**Jad Al Hassan**

202400472

**Issam Fawaz**

202400437

*December 7, 2025*

*Submitted to instructor Dr. Khaleel Merchad in fulfillment of the requirements of  
phase 2 of the database project for the course CSC375 – Database Management  
System*

# Table of Contents

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>1. Introduction.....</b>                                        | <b>3</b>  |
| <b>2. Database Requirements.....</b>                               | <b>4</b>  |
| <b>3. E-R Model.....</b>                                           | <b>7</b>  |
| ➤ <i>ER Diagram Notations: .....</i>                               | <i>7</i>  |
| ➤ <i>Full ER Model for Esports Team Performance Tracker: .....</i> | <i>8</i>  |
| ➤ <i>ER Model Breakdown: .....</i>                                 | <i>9</i>  |
| <b>3.1 Entity Types and Their Attributes.....</b>                  | <b>9</b>  |
| <b>3.2 Relationships and their Explanation .....</b>               | <b>16</b> |
| <b>4. Relational Model.....</b>                                    | <b>22</b> |
| <b>5. Database Implementation .....</b>                            | <b>23</b> |
| ➤ <i>DDL Queries:.....</i>                                         | <i>23</i> |
| ➤ <i>DML Queries: .....</i>                                        | <i>37</i> |
| <b>6. Basic Queries .....</b>                                      | <b>39</b> |
| <b>7. Advanced Queries.....</b>                                    | <b>51</b> |
| <b>8. Conclusion .....</b>                                         | <b>63</b> |

# 1. Introduction

In the rapidly growing world of competitive gaming, esports organizations are responsible to manage players, coaches, and tournaments across a variety of games and leagues. As teams compete in global events in games such as League of Legends, Valorant, and CS:GO, the amount of data generated from player performances, match outcomes, and training activities continues to expand. Efficiently organizing and analyzing this information has become essential for teams that want to improve performance, strategize, and maintain a competitive edge. This growing complexity has created a strong need for structured systems that can accurately record and track performance data across multiple dimensions of esports management.

This Esports Team Performance Tracker serves as a centralized system to monitor every aspect of team and player activity. A system like this supports organizations in keeping detailed records of tournament results, player statistics, training sessions, and coaching feedback. By using this data, team managers and analysts can identify performance trends, evaluate player development, and make decisions about team composition and strategy based on numbers. Also, coaches can use performance insights to plan more effective training sessions and address weaknesses before official competitions.

Beyond improving performance analysis, the use of a tracking system represents a step toward data-driven management in esports. By recording player statistics, match histories, and training outcomes, organizations can evaluate progress with measurable indicators rather than subjective impressions. This promotes more strategic decision-making, allowing teams to use their resources effectively, measure the effects of their current lineups, and design evidence-based training programs. As esports continues to evolve into a structured and analytical field, such systems play an important role in bridging the gap between intuition and data-backed strategy.

## 2. Database Requirements

The Esports Team Performance Tracker is designed to support organizations that manage professional esports teams competing in various online games and tournaments. The system is intended to monitor team performance, individual player statistics, match outcomes, tournament participation, training sessions, and in-game analytics across different games and events. This centralized database allows team managers, coaches, analysts, and players to access accurate, up-to-date performance data and make informed decisions about strategy and training.

At the heart of the system are teams, each uniquely identified by a team ID and associated with a team name, region, founding date, and current ranking. A team participates in multiple online tournaments, and each tournament has attributes such as tournament ID, name, host organization, game title (e.g., League of Legends, Valorant, CS:GO), prize pool, start, and end dates. Tournaments may be recurring, and each occurrence is tracked as a separate instance for statistical and historical purposes.

Each player in the system has a unique player ID, gamer tag, full name, country, date of birth, role (e.g., support, carry, in-game leader), and date joined the team. Players belong to one and only one team at a time, but they may change teams over time, which requires a historical log of player-team associations. This relationship includes start and end dates, ensuring that the system can retrieve the composition of a team at any point in time. Players can also be marked as active or inactive (e.g., benched, busy, or retired).

The core of performance tracking involves matches. A match is identified by a match ID and is associated with a specific tournament and round (e.g., quarterfinals, semifinals), a scheduled date and time, and whether it is a best-of-1, best-of-3, or best-of-5 series. A match involves two teams: a home team and an opponent team. The results of each match include who won, the score line, and duration of the game. The system must also track the individual performance statistics of each player in a match,

such as kills, assists, deaths, damage dealt, accuracy, healing, and objective contributions, depending on the game being played.

To support training and preparation, the system also manages scrimmages (practice matches) and training sessions. Each training session has a training ID, date, duration, and type (e.g., aim training, team coordination, strategy planning), and is linked to one or more players. Scrimmages are logged similarly to official matches but are marked as “non-official” and may or may not be recorded in public records. Both training and scrimmage data contribute to tracking player development over time.

Another essential aspect is coaches and analysts. Each staff member has a staff ID, name, role (coach, assistant coach, analyst), and the team they are affiliated with. They oversee the training sessions, design strategies, and analyze performance. The system keeps track of which staff member supervised each training session and optionally links feedback or evaluation notes to individual players.

The system supports a wide range of data operations. For example, analysts can query the system to retrieve a player's average kills per game over the last five matches, compare team performance before and after a roster change, or generate reports summarizing tournament results. Coaches can review participation in training sessions, assess improvement trends, and identify underperforming players.

Common queries include:

- “List all tournaments in which Team Alpha participated in 2025.”
- “Display the match history of PlayerX across all games.”
- “Show team composition changes over time for Team Bravo.”
- “Retrieve the KDA ratio (Kills/Deaths/Assists) of a player for the past three months.”
- “Generate win-loss statistics for each team by game title.”

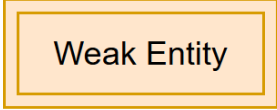
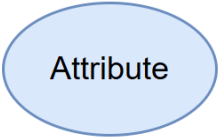
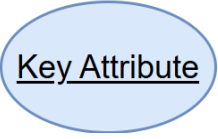
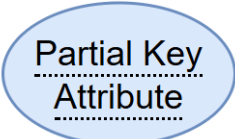
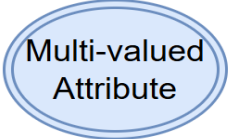
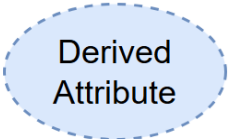
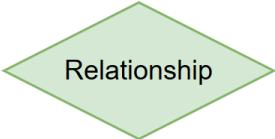
Triggers can be added to ensure data consistency. For instance, a trigger can prevent entering match statistics for a player not officially part of a team during that match. Another trigger may notify the coach when a player misses three or more consecutive training sessions.

Stored procedures can be used to calculate and store derived statistics, such as overall team win rate, average player accuracy per month, or the most used strategies based on match records. Functions might be used to compute the impact rating of a player, factoring kills, assists, damage, and deaths into a weighted score.

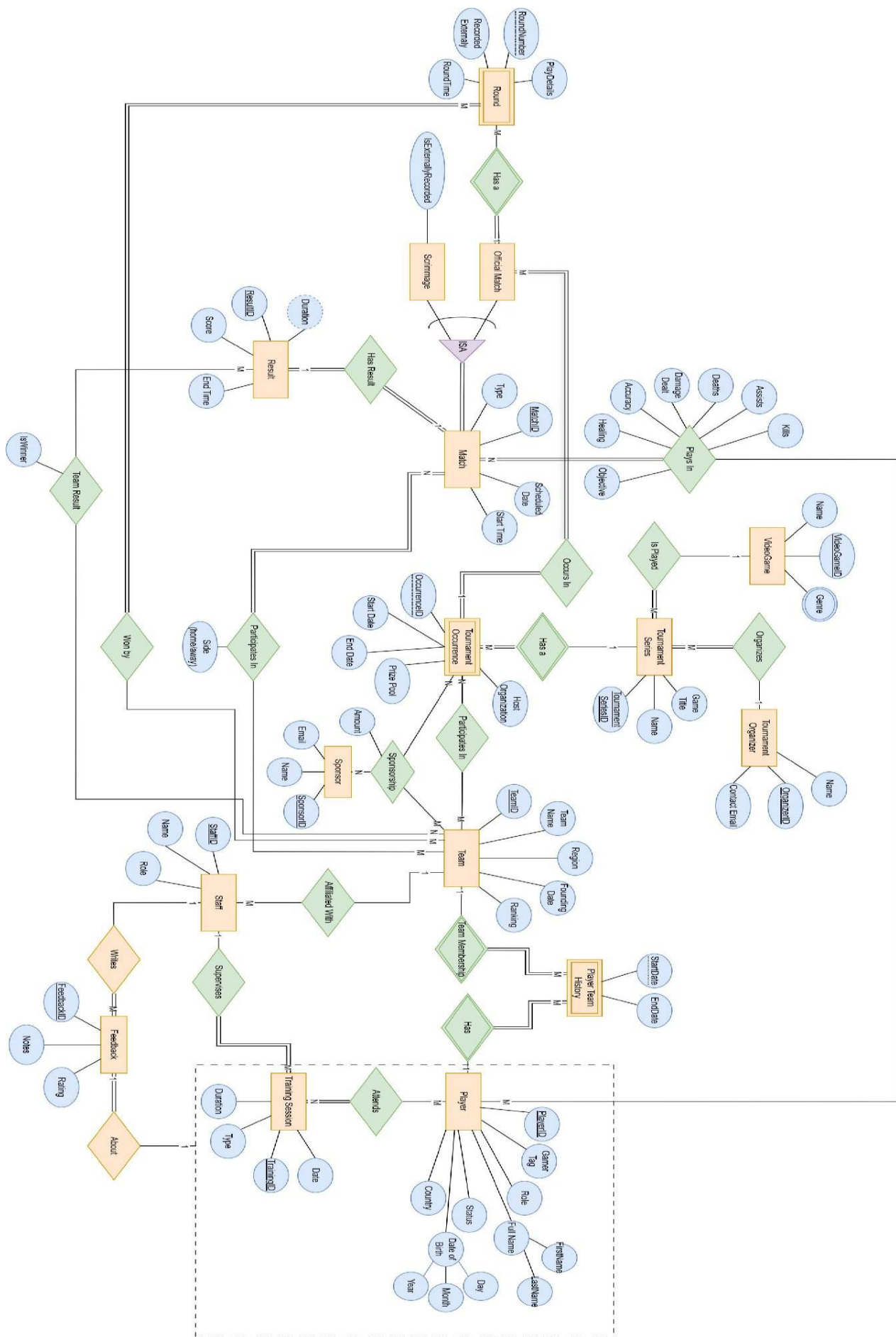
This database system forms a robust backend for managing an esports organization's competitive and training performance. It provides a foundation for performance analysis, decision-making, team development, and long-term growth in the dynamic field of competitive gaming.

### 3. E-R Model

➤ ER Diagram Notations:

| Symbol/Shape                                                                        | ER Component             | Description                      |
|-------------------------------------------------------------------------------------|--------------------------|----------------------------------|
|    | Entity                   | Rectangle                        |
|    | Weak Entity              | Double Bordered Rectangle        |
|    | Attribute                | Oval                             |
|    | Key Attribute            | Oval with Underlined Text        |
|  | Partial Key Attribute    | Oval with Dashed Underlined Text |
|  | Multi-valued Attribute   | Double Border Oval               |
|  | Derived Attribute        | Dashed Border Oval               |
|  | Relationship             | Diamond                          |
|  | Identifying Relationship | Double Bordered Diamond          |

➤ Full ER Model for Esports Team Performance Tracker:

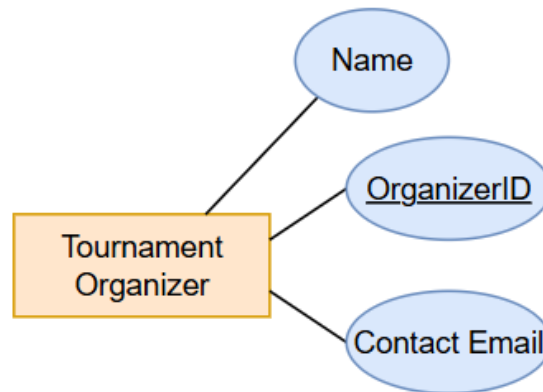




## ➤ ER Model Breakdown:

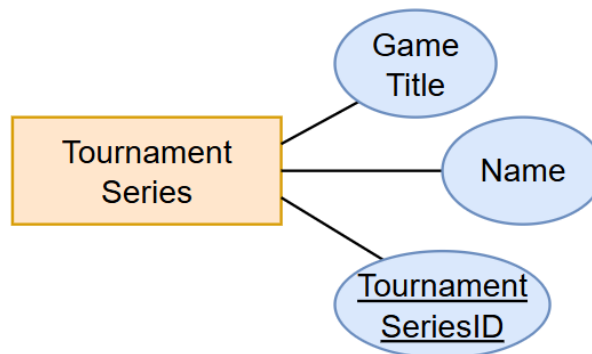
### 3.1 *Entity Types and Their Attributes*

#### 3.1.1 Tournament Organizer Entity



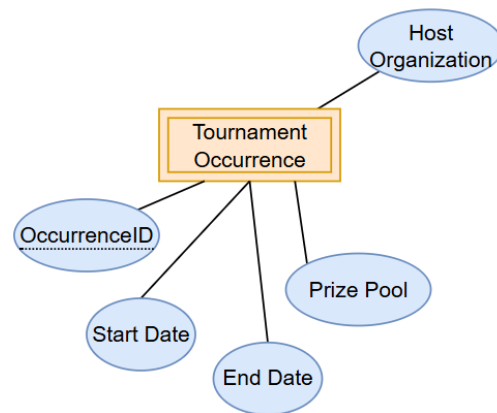
Every tournament is backed by a dedicated organizer, which is why the TournamentOrganizer entity plays a crucial role in our system. Each organizer is a distinct entity identified by a unique OrganizerID, ensuring clear ownership and management of tournaments. We store the organizer's Name and ContactEmail to maintain proper communication channels with hosting organizations. This information helps teams, sponsors, and staff stay connected with official event hosts and ensures that every tournament can be accurately traced back to its managing body.

#### 3.1.2 Tournament Series Entity



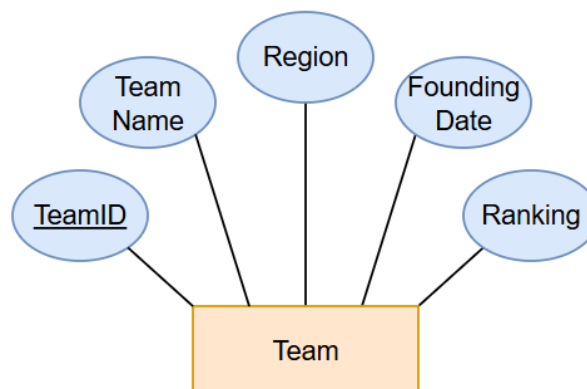
The TournamentSeries entity represents a recurring competition, such as the “Worlds Championship” or “Valorant Champions Tour.” Each series is uniquely identified by a TournamentSeriesID (primary key) and includes a Name to distinguish it from others. The GameTitle attribute specifies which video game the series is based on, ensuring that each competition is tied to a specific game. This entity provides continuity for tournaments that occur annually or seasonally under the same brand, allowing the system to organize multiple occurrences of the same event over time.

### 3.1.3 Tournament Occurrence Entity



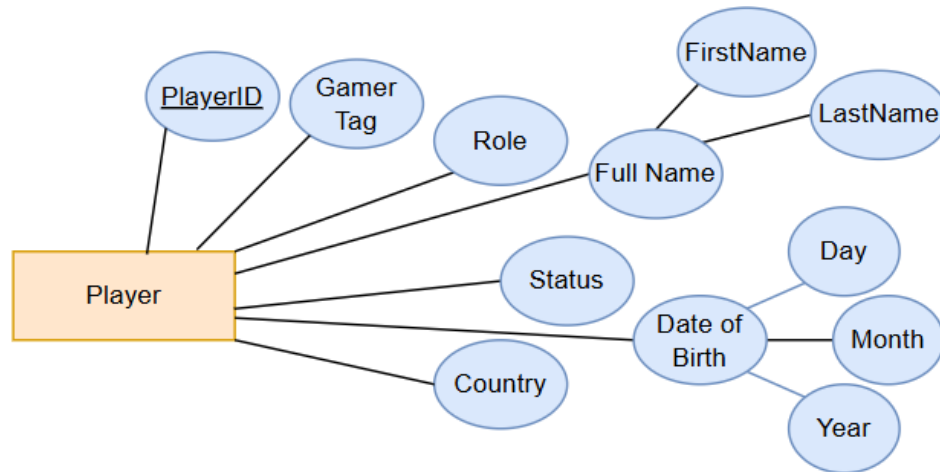
The TournamentOccurrence entity represents a specific edition or instance of a tournament series. For example, Valorant Champions Tour 2025 or League of Legends Worlds 2024. It is modeled as a weak entity because each occurrence depends on its parent TournamentSeries for identification. The OccurrenceID serves as its weak key attribute, uniquely identifying each occurrence within its series. Additional attributes such as HostOrganization, StartDate, EndDate, and PrizePool describe the details of that particular edition, including who hosted it, when it took place, and the total rewards offered. This design ensures that every tournament occurrence is historically tracked while remaining linked to its series.

### 3.1.4 Team Entity



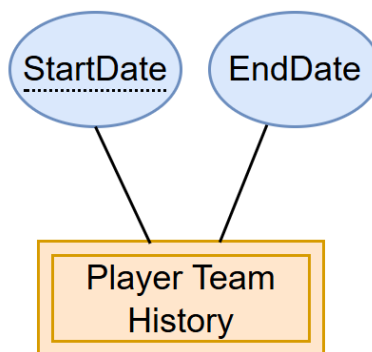
The Team entity represents each professional esports team managed within the system. Every team is uniquely identified by a TeamID (primary key) and includes key attributes such as TeamName, Region, FoundingDate, and Ranking. These details capture the identity, origin, and performance of the team. By maintaining this information, the system can accurately track each team's evolution, competitive record, and participation across tournaments, ensuring that historical and current data about every organization are easily accessible for analysis and reporting.

### 3.1.5 Player Entity



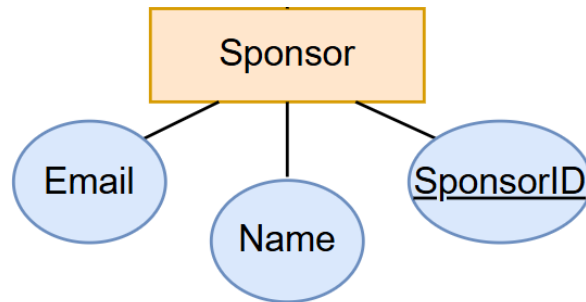
The Player entity represents each professional esports player tracked by the system. Every player is uniquely identified by a PlayerID (primary key) and is described by attributes such as GamerTag, FullName(which is a composite attribute consisting of FirstName and LastName), Country, Role, Status, and DateOfBirth(which is made up of Day, Month, and Year). These attributes capture both personal and competitive information, helping distinguish players across teams and tournaments. By storing this data, the system can monitor individual performance statistics, track roster changes over time, and support detailed player analysis essential for performance evaluation and team management.

### 3.1.6 Player Team History Entity



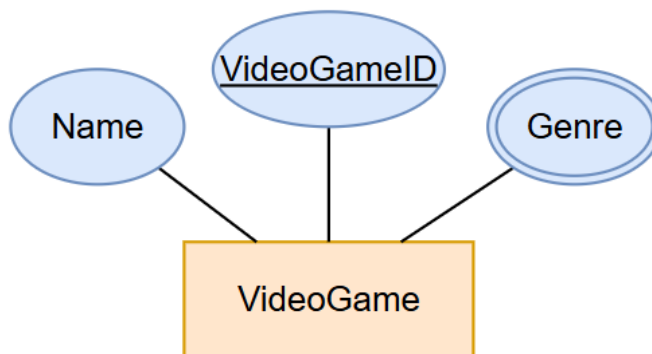
The PlayerTeamHistory entity records the history of player memberships across different teams over time. It is modeled as a weak entity, since each record depends on both the Player and Team entities for identification. Its partial key, StartDate, distinguishes each membership period for the same player and team combination, while EndDate indicates when that association ended. This structure ensures that the system can retrieve past rosters, track player transfers, and determine the composition of any team at a specific point in time, maintaining an accurate historical record of player movements within the esports ecosystem.

### 3.1.7 Sponsor Entity



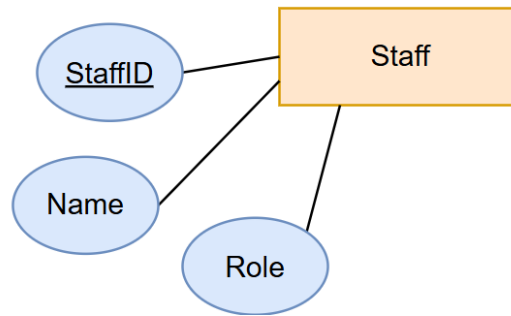
The Sponsor entity represents the organizations or brands that provide financial or promotional support to esports teams for a certain tournament occurrence. Each sponsor is uniquely identified by a SponsorID (primary key) and includes attributes such as Name and Email to store essential business contact information. Sponsors play a vital role in funding teams during specific tournaments or seasons, helping sustain competitive operations. By maintaining sponsor details, the system ensures that sponsorship agreements, financial contributions, and partnerships can be accurately recorded, tracked, and linked to team performance and tournament participation.

### 3.1.8 Video Game Entity



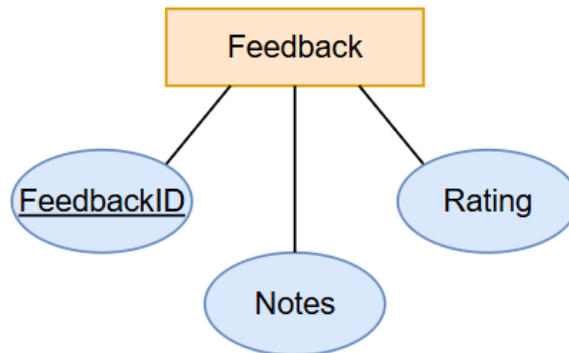
The VideoGame entity represents the game titles featured in tournaments and tracked within the system. Each video game is uniquely identified by a VideoGameID (primary key) and includes attributes such as Name, Creator, and Genre. The Genre attribute is multivalued, since a game can belong to multiple categories (e.g., ‘First-Person Shooter’ and ‘Tactical’). This entity ensures that every tournament, match, and player performance is clearly associated with the correct game, allowing the system to organize and compare data across different games.

### 3.1.9 Staff Entity



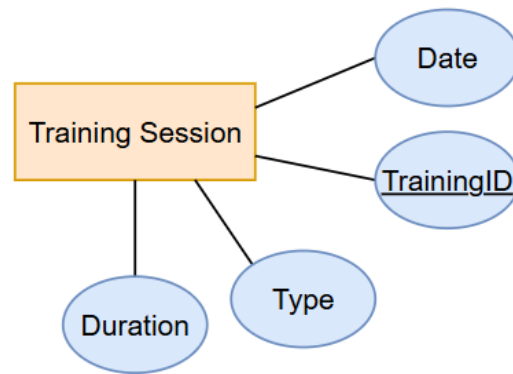
The Staff entity represents the people responsible for coaching, analysis, and team management within an esports organization. Each staff member is uniquely identified by a StaffID (primary key) and includes attributes such as Name and Role, which specify their position (such as coach, assistant coach, or analyst). This entity allows the system to record which staff members oversee training sessions, supervise player development, and provide feedback. By tracking staff information separately from players, the system can link performance improvements and training outcomes to specific coaches or analysts, promoting accountability and structured team management.

### 3.1.10 Feedback Entity



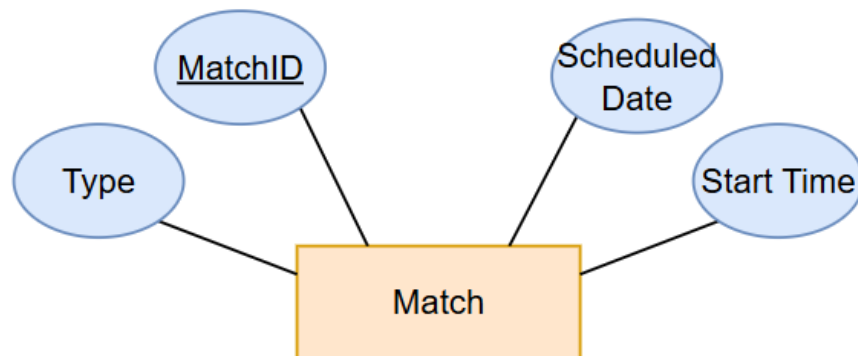
The Feedback entity contains evaluations or comments made by staff members about players and their performance during training sessions. Each feedback entry is uniquely identified by a FeedbackID (primary key) and includes attributes such as Notes and Rating. It is linked through aggregation to the relationship between Staff and TrainingSession, since feedback is given by a staff member who supervises a specific session. This design ensures that feedback exists only in the context of an actual training interaction, allowing the system to track player progress, monitor coaching effectiveness, and maintain detailed records assessments.

### 3.1.11 Training Session Entity



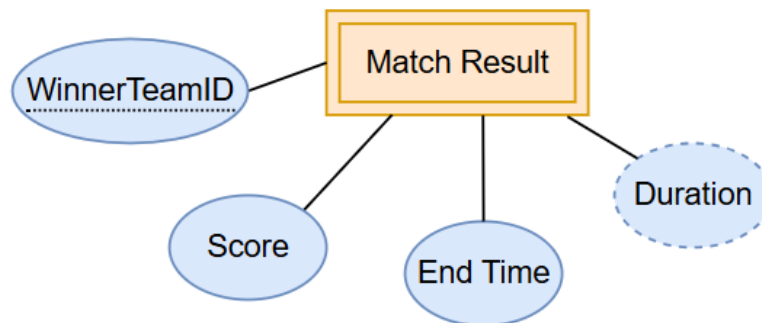
The TrainingSession entity represents the practice sessions organized to improve team performance and coordination. Each training session is uniquely identified by a TrainingID (primary key) and includes attributes such as Type, Date, and Duration. This entity allows the system to record which players attended each session and which staff members supervised it, enabling detailed analysis of training frequency, attendance, and its correlation with player or team performance over time.

### 3.1.12 Match Entity



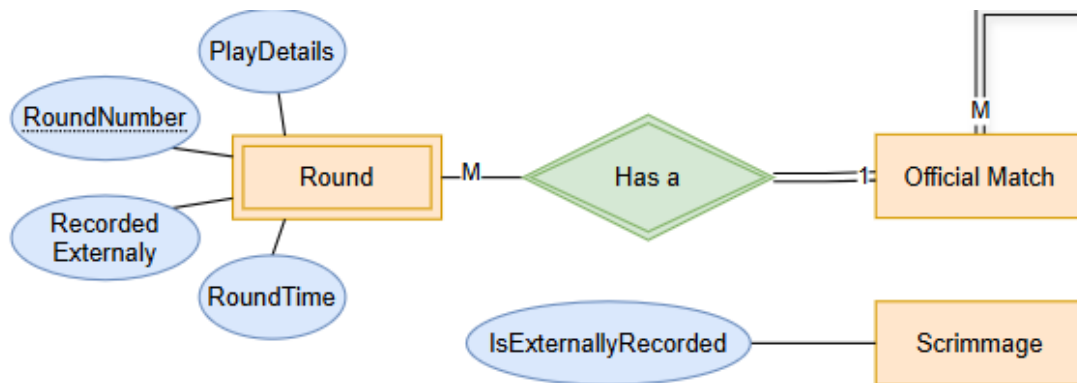
The Match entity represents an encounter between two teams within a tournament or during a scrimmage. Each match is uniquely identified by a MatchID (primary key) and includes attributes such as the Type of the match, the ScheduledDate, and the StartTime. The Match entity serves as a super type in an ISA hierarchy, with OfficialMatch and Scrimmage as its subtypes. This allows the system to distinguish between tournament matches and practice matches. This design enables accurate tracking of results, player performance, and tournament participation, while also supporting analyses that compare official performance with training outcomes.

### 3.1.13 Match Result Entity



The MatchResult entity stores the outcome details of each match played in the system. It is uniquely associated with a Match through a one-to-one relationship and includes attributes such as Scoreline, Duration, and WinnerTeamID. The WinnerTeamID serves as a foreign key referencing the winning team, ensuring each result is traceable to its victor. The Duration attribute is derived from the match's start and end time. This entity allows the system to summarize match outcomes, generate team win/loss statistics, and support performance analytics across tournaments and scrimmages.

### 3.1.14 Official Match and Scrimmage Entities



The OfficialMatch and Scrimmage entities are subtypes of the Match entity, used to distinguish between official tournament games and practice matches. OfficialMatch instances occur within tournaments and are linked to TournamentOccurrence, ensuring all competitive results are tied to official events. Also, each official match has Rounds related to. Scrimmage represents non-official practice matches that help teams improve strategies and performance outside tournament settings.

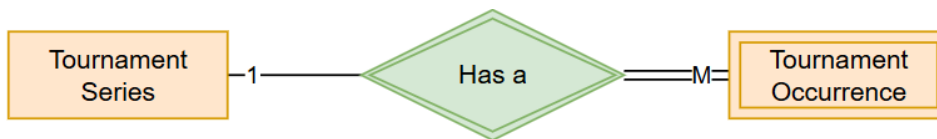
## 3.2 Relationships and their Explanation

### 3.2.1 Tournament Organizer organizes Tournament Series



Links each tournament series to its hosting organization. One organizer can manage many series, while each series must have exactly one organizer, hence the total participation. This makes it clear what organization organized a certain tournament.

### 3.2.2 Tournament Series has a Tournament Occurrence



Connects each occurrence (edition) to its parent series. It's an identifying relationship: an occurrence can't exist without a series (total participation on Occurrence), and is uniquely determined by the series plus its weak key. Each tournament series could have many occurrences.

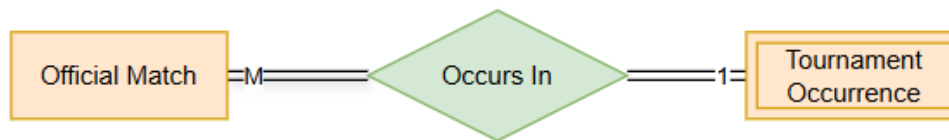
### 3.2.3 Video Game is played during Tournament Series



States which game a series is run on (e.g., League of Legends, Valorant, etc.). Enables grouping results and stats by game title. Each tournament series must have one video game being played and each game could be played in multiple tournament series.



### 3.2.4 Official Match occurs in Tournament Occurrence



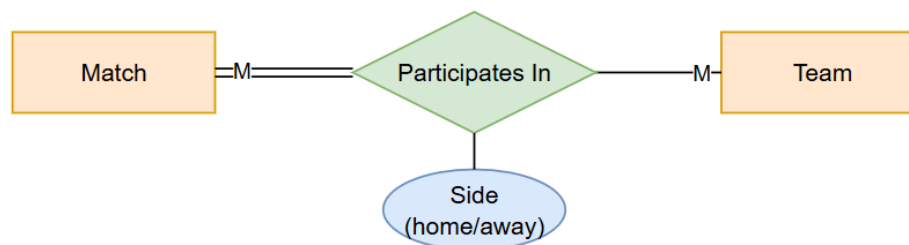
Associates every official match with exactly one tournament occurrence. This relates to the Official Match subtype (not on Match) so scrimmages remain outside tournaments. Each official match must be in an occurrence of a tournament and each occurrence should have matches therefore the total participation on both sides.

### 3.2.5 Match has Rounds



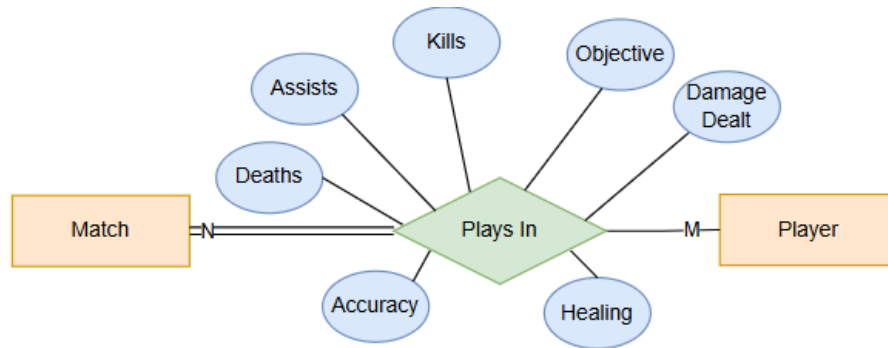
This relationship breaks down each match into its individual rounds. Round is a weak entity that depends on Match for identification, using a composite key (MatchID, RoundNumber). The participation is total on Round, since every round must belong to exactly one match, and total on Match if since match includes at least one round. This structure supports flexible formats such as best-of-1, best-of-3, or best-of-5 series, allowing the system to store detailed round-level outcomes.

### 3.2.6 Team participates in Match



This relationship records each team's involvement in a match and includes attributes such as Side (Home/Away). For OfficialMatch, the business rule enforces that exactly two teams participate in each match, each with a unique side designation. This ensures clear competitive structure and prevents data inconsistencies, while supporting detailed analyses such as head-to-head results. Each match must have 2 teams participating.

### 3.2.7 Player plays in Match



This relationship captures player involvement per match with stats on the relationship (Kills, Assists, Deaths, DamageDealt, Accuracy, Healing, Objective). This supports KDA, Last-N games, and time-window performance queries. Each match must have players and each player could or could not compete in multiple matches,

### 3.2.8 Match has a Match Result



This identifying relationship links each Match to its MatchResult, making that a result cannot exist without the match. The participation is total on both sides since every match must have one result once completed, and every result must belong to exactly one match. The cardinality is 1:1, reflecting that each match produces only one set of outcome details.

### 3.2.9 Round is won by Team



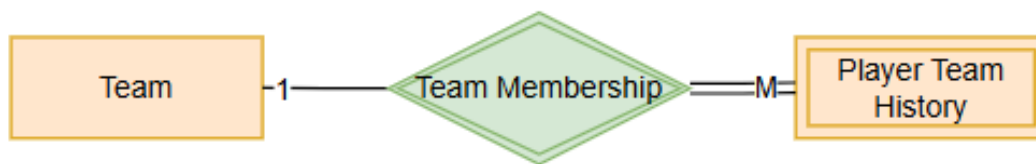
This relationship records which Team wins each Round, allowing the system to capture results at the round level. It is important for getting the overall match score lines from individual rounds which enables detailed, round-by-round performance analysis. By tracking round winners separately, the database can support statistics such as round win rates.

### 3.2.10 Player has Player Team History



This identifying relationship connects each Player to their corresponding records in PlayerTeamHistory, ensuring that every membership entry is tied to a specific player. The participation is total on the PlayerTeamHistory side, since no history record can exist without a player, and 1: N from Player, as one player can have multiple historical entries over time. This relationship contributes the PlayerID to the weak entity's composite key, allowing the system to accurately trace all past and present team associations for each player.

### 3.2.11 Team team membership Player Team History



This identifying relationship links each Team to its corresponding PlayerTeamHistory records, defining which players have been part of the team over time. The participation is total on the PlayerTeamHistory side, since every history record must belong to a specific team, and 1: N from Team, as one team can have multiple player memberships throughout its existence. This relationship contributes the TeamID to the weak entity's composite key, which together with PlayerID and the StartDate partial key uniquely identifies each player's membership period within a team.

### 3.2.12 Staff supervises Training Session



Connects a Staff member to the TrainingSession they run. Cardinality is 1:N (one staff can supervise many sessions). Every Training Session must have a staff present therefore the participation constraint.

### 3.2.13 Player attends Training Session



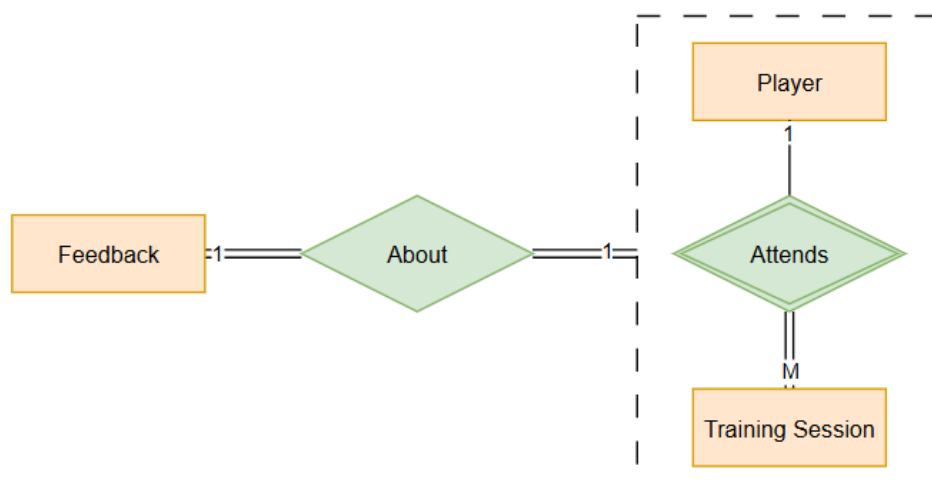
A many-to-many relationship between Player and TrainingSession. For a session to hold there must be at least one player present. This relationship shows which training sessions a player has attended. This is what the staff actually writes feedback on.

### 3.2.14 Staff writes Feedback



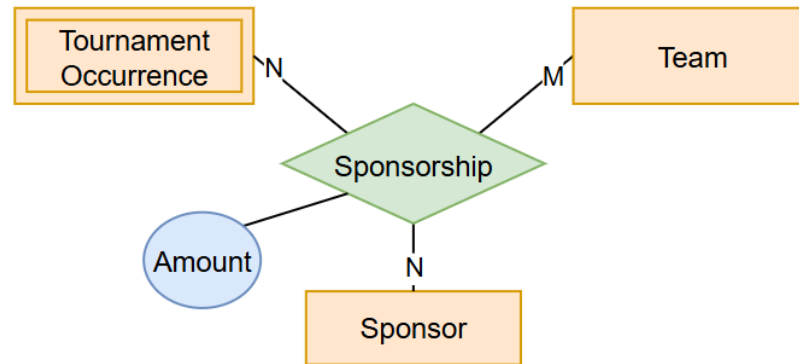
Shows who writes each feedback entry. Cardinality is 1: N as one staff can write many feedbacks. Participation is total on Feedback (every feedback must be written by exactly one staff member) and partial on Staff. This enforces accountability and enables effectiveness reporting.

### 3.2.15 Feedback about Aggregation of Player attends Training Session



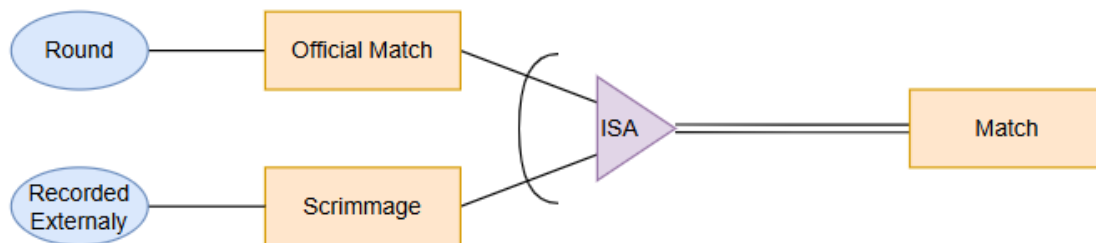
This relationship relates each feedback written by the staff to the whole training session (Player attends Training Session). Participation is total on both sides as each feedback needs to be about something and each training session must have the feedback on it so that the teams know what they need to improve.

### 3.2.16 Team is sponsored by Sponsor for certain Occurrence (ternary relationship)



Models real-world deals: a Sponsor backs a Team during a specific Tournament Occurrence with the amount on the relationship. In the Sponsorship relationship, all sides have partial participation, meaning teams, sponsors, and tournament occurrences can exist independently of sponsorship deals. The relationship is many-to-many-to-many (N:N:N) meaning a team may have several sponsors in a tournament, a sponsor can support multiple teams, and the same team/sponsor pair can appear across different occurrences.

### 3.2.17 Match ISA Official Match or a Scrimmage



The ISA relationship between Match, OfficialMatch, and Scrimmage is used to represent specialization within the general concept of a match. The Match entity acts as the supertype, storing shared attributes such as match ID, start and end time, and duration. It then divides into two subtypes: OfficialMatch, representing tournament matches, and Scrimmage representing practice or non-official matches. This ISA relationship is disjoint, since a match can only be one type at a time, and typically total, meaning every match is either official or a scrimmage. Modeling it this way allows the system to cleanly separate competitive and practice data while maintaining a common match structure for performance tracking and analytics.

## 4. Relational Model

- TournamentOrganizer (OrganizerID, Name, ContactEmail)
- TournamentSeries (TournamentSeriesID, Name, GameTitle, #OrganizerID **NOT NULL**, #VideoGameID **NOT NULL**)
- VideoGame (VideoGameID, Name)
- Genres (#VideoGameID, Genre)
- TournamentOccurrence (OccurrenceID, #TournamentSeriesID, StartDate, EndDate, PrizePool, HostOrganization)
- Team (TeamID, TeamName, Region, FoundingDate, Ranking)
- TournamentParticipation (#TeamID, #OccurrenceID, #TournamentSeriesID)
- Player (PlayerID, GamerTag, Role, FirstName, LastName, Status, Day, Month, Year, Country)
- PlayerTeamHistory (StartDate, #TeamID, #PlayerID, EndDate)
- OfficialMatch (MatchID, Type, ScheduledDate, StartTime, Round, #OccurrenceId **NOT NULL**, #TournamnetSeriesID **NOT NULL**)
- Scrimmage (MatchID, Type, ScheduledDate, StartTime, isExternallyRecorded)
- TeamMatchParticipation (#TeamID, #MatchID, Side{Home/Away})
- Round (RoundNumber, #MatchID, PlayDetails, RoundTime)
- WonBy(#RoundNumber, #TeamID)
- PlayerMatchStats (#PlayerID, #MatchId, Kills, Assists, Deaths, DamageDealt, Accuracy, Healing, Objective)
- MatchResult (ResultID, Score, EndTime, #MatchID **NOT NULL**) - *we did not include Duration because its derived*
- TeamResult (#ResultID, #TeamID, isWinner)
- Sponsor (SponsorID, Name, Email)
- Sponsorship (#SponsorID, #OccurrenceID, #TournamnetSeriesID, #TeamID, Amount)
- Staff (StaffID, Name, Role, #TeamID)
- TrainingSession (TrainingID, Date, Type, Duration, #StaffID **NOT NULL**)
- PlayerAttendsTrainingSession (#PlayerID, #TrainingID)
- Feedback (FeedbackID, Notes, Rating, #StaffID **NOT NULL**, #PlayerID **NOT NULL**, #TrainingID **NOT NULL**)

## 5. Database Implementation

To create and query our database, we used MySQL.

We started by first creating the tables, defining and implementing all the constraints that exist in your DB, including foreign keys, **NOT NULL**, **DEFAULT**, **CHECK**, etc.

Then we populated our Database using random data and fulfilling the required number of data per table.

### ➤ DDL Queries:

#### 5.1.1 Tournament Organizer

```
create table TournamentOrganizer
(
  OrganizerID    varchar(10),
  Name           varchar(50) not null,
  ContactEmail   varchar(100),
  primary key (OrganizerID)
);
```

This table stores information about tournament organizers. OrganizerID is the primary key that every organizer is uniquely identified by. Name is NOT NULL because an organizer must have a name. ContactEmail is optional.

### 5.1.2 VideoGame

```
create table VideoGame
  (VideoGameID    varchar(10),
   Name           varchar(50) not null,
   primary key (VideoGameID)
  );
```

Stores information about the video games used in tournament series. VideoGameID is the primary key, and Name is required since each game must have a title.

### 5.1.3 Genres

```
create table Genres
  (VideoGameID    varchar(10),
   Genre          varchar(30),
   primary key (VideoGameID, Genre),
   foreign key (VideoGameID) references VideoGame (VideoGameID)
     on delete cascade
  );
```

---

This table connects video games with their genres. The primary key is a combination of VideoGameID and Genre. The foreign key ensures the game exists and cascades deletions so genres disappear when the game is removed.



#### 5.1.4 TournamentSeries

```
create table TournamentSeries
(
    TournamentSeriesID varchar(10),
    Name                varchar(50) not null,
    GameTitle           varchar(50),
    OrganizerID         varchar(10) not null,
    VideoGameID         varchar(10) not null,
    primary key (TournamentSeriesID),
    foreign key (OrganizerID) references TournamentOrganizer (OrganizerID)
        on delete cascade,
    foreign key (VideoGameID) references VideoGame (VideoGameID)
        on delete cascade
);
```

This table represents a long-term series of tournaments. TournamentSeriesID is the primary key. OrganizerID and VideoGameID are NOT NULL and are foreign keys, ensuring every series is linked to an organizer and a video game. Cascading deletion cleans up related series if the parent is removed.

#### 5.1.5 TournamentOccurrence

```
create table TournamentOccurrence
(
    OccurrenceID        varchar(10),
    TournamentSeriesID  varchar(10) not null,
    StartDate           date not null,
    EndDate             date,
    PrizePool           numeric(12,2) check (PrizePool >= 0),
    HostOrganization    varchar(100),
    primary key (OccurrenceID),
    foreign key (TournamentSeriesID) references TournamentSeries (TournamentSeriesID)
        on delete cascade
);
```

Stores each specific occurrence of a tournament series. OccurrenceID is the primary key. StartDate is required. TournamentSeriesID is a foreign key with CASCADE because an occurrence cannot exist without the series.

### 5.1.6 Team

```
create table Team
(
    TeamID          varchar(10),
    TeamName        varchar(50) not null,
    Region          varchar(30),
    FoundingDate    date,
    Ranking          numeric(4,0) check (Ranking >= 0),
    primary key (TeamID)
);
```

This table contains information about the teams participating in tournaments. TeamID is the primary key. TeamName is mandatory. Ranking has a CHECK to ensure its non-negative.

### 5.1.7 TournamentParticipation

```
create table TournamentParticipation
(
    TeamID          varchar(10),
    OccurrenceID    varchar(10),
    TournamentSeriesID varchar(10),
    primary key (TeamID, OccurrenceID, TournamentSeriesID),
    foreign key (TeamID) references Team (TeamID)
        on delete cascade,
    foreign key (OccurrenceID) references TournamentOccurrence (OccurrenceID)
        on delete cascade,
    foreign key (TournamentSeriesID) references TournamentSeries (TournamentSeriesID)
        on delete cascade
);
```

Tracks which teams join which tournament occurrence. The composite primary key (TeamID, OccurrenceID, TournamentSeriesID) prevents duplicates. All three attributes are foreign keys with CASCADE because participation depends on both the team and the tournament.

### 5.1.8 Player

```
create table Player
(
    PlayerID      varchar(10),
    GamerTag      varchar(30) not null,
    Role          varchar(30),
    FirstName     varchar(30),
    LastName      varchar(30),
    Status        varchar(20),
    Day           numeric(2,0) check (Day >= 1 and Day <= 31),
    Month         numeric(2,0) check (Month >= 1 and Month <= 12),
    Year          numeric(4,0),
    Country       varchar(40),
    primary key (PlayerID)
);
```

This table stores player information. PlayerID is the PK. GamerTag is required. Day and Month have CHECK constraints to ensure valid dates. Everything else is optional to allow flexibility in player profiles. Day/Month/Year are written that way as they were modelled as a composite attribute in our ER Diagram and relational model.

### 5.1.9 PlayerTeamHistory

```
create table PlayerTeamHistory
(
    StartDate     date,
    TeamID        varchar(10),
    PlayerID      varchar(10),
    EndDate       date,
    primary key (PlayerID, TeamID, StartDate),
    foreign key (TeamID) references Team (TeamID)
        on delete cascade,
    foreign key (PlayerID) references Player (PlayerID)
        on delete cascade
);
```

Keeps track of when a player joined or left a team. Composite PK (PlayerID, TeamID, StartDate) makes each team period unique. Both TeamID and PlayerID are foreign keys with CASCADE because history cannot exist without them.

### 5.1.10 OfficialMatch

```
create table OfficialMatch
(
    MatchID          varchar(10),
    Type             varchar(20),
    ScheduledDate     date,
    StartTime         time,
    Round            varchar(20),
    OccurrenceID      varchar(10) not null,
    TournamentSeriesID varchar(10) not null,
    primary key (MatchID),
    foreign key (OccurrenceID) references TournamentOccurrence (OccurrenceID)
        on delete cascade,
    foreign key (TournamentSeriesID) references TournamentSeries (TournamentSeriesID)
        on delete cascade
);
```

Represents official tournament matches. MatchID is the PK. OccurrenceID and TournamentSeriesID are NOT NULL foreign keys because every official match belongs to a specific event. CASCADE removes matches if the event is deleted.

### 5.1.11 Scrimmage

```
create table Scrimmage
(
    MatchID          varchar(10),
    Type             varchar(20),
    ScheduledDate     date,
    StartTime         time,
    IsExternallyRecorded numeric(1,0)
        check (IsExternallyRecorded in (0,1)),
    primary key (MatchID)
);
```

Stores practice matches. MatchID is the PK. IsExternallyRecorded uses a CHECK constraint to only allow 0 or 1 (1 means true and 0 means false). No foreign keys since scrimms are made to be independent.

### 5.1.12 TeamMatchParticipation

```
create table TeamMatchParticipation
(
    TeamID          varchar(10),
    MatchID         varchar(10),
    Side            varchar(5)
        check (Side in ('Home','Away')),
    primary key (TeamID, MatchID),
    foreign key (TeamID) references Team (TeamID)
        on delete cascade,
    foreign key (MatchID) references OfficialMatch (MatchID)
        on delete cascade
);
```

Shows which teams played in each match, with their side (Home/Away). Composite PK of TeamID and MatchID prevents duplicates. TeamID is a foreign key, and MatchID references OfficialMatch only. The CHECK on Side ensures only Home or Away are valid value.

### 5.1.13 Round

```
create table Round
(
    RoundNumber     numeric(3,0),
    MatchID         varchar(10),
    PlayDetails     varchar(200),
    RoundTime       numeric(3,0), -- Time it took to finish a round in minutes.
    primary key (RoundNumber, MatchID),
    foreign key (MatchID) references OfficialMatch (MatchID)
        on delete cascade
);
```

Stores individual rounds inside a match. Composite PK (RoundNumber, MatchID) uniquely identifies each round within a match. RoundTime is the duration in minutes. MatchID must reference an existing official match.

#### 5.1.14 WonBy

```
create table WonBy
(
    RoundNumber    numeric(3,0),
    MatchID        varchar(10),
    TeamID         varchar(10),
    primary key (RoundNumber, MatchID, TeamID),
    foreign key (RoundNumber, MatchID) references Round (RoundNumber, MatchID)
        on delete cascade,
    foreign key (TeamID) references Team (TeamID)
        on delete cascade
);
```

Indicates which team won a round. Composite PK (RoundNumber, MatchID, TeamID) prevents duplicates. Foreign keys ensure the round and team actually exist.

#### 5.1.15 PlayerMatchStats

```
create table PlayerMatchStats
(
    PlayerID       varchar(10),
    MatchID        varchar(10),
    Kills          numeric(4,0) check (Kills >= 0),
    Assists        numeric(4,0) check (Assists >= 0),
    Deaths        numeric(4,0) check (Deaths >= 0),
    DamageDealt    numeric(10,0) check (DamageDealt >= 0),
    Accuracy       numeric(5,2) check (Accuracy >= 0 and Accuracy <= 100),
    Healing        numeric(10,0) check (Healing >= 0),
    Objective      varchar(200),
    primary key (PlayerID, MatchID),
    foreign key (PlayerID) references Player (PlayerID)
        on delete cascade,
    foreign key (MatchID) references OfficialMatch (MatchID)
        on delete cascade
);
```

Contains per-player performance statistics. Composite PK (PlayerID, MatchID) ensures each player only has one set of stats per match. All numeric attributes have CHECKs to keep values

valid (no negative kills, accuracy between 0–100). Objective is text describing the player's task during a match.

#### 5.1.16 MatchResult

```
create table MatchResult
(
    ResultID      varchar(10),
    Score         varchar(20),
    EndTime       time,
    MatchID       varchar(10) not null,
    primary key (ResultID),
    foreign key (MatchID) references OfficialMatch (MatchID)
        on delete cascade
);
```

Represents the final result of a match. ResultID is the PK. MatchID is NOT NULL and references an existing official match. CASCADE ensures the result is deleted automatically if the match is removed.

### 5.1.17 TeamResult

```
create table TeamResult
(
    ResultID      varchar(10),
    TeamID        varchar(10),
    IsWinner      numeric(1,0)
                check (IsWinner in (0,1)), -- 1 means winner and 0 means loser
    primary key (ResultID, TeamID),
    foreign key (ResultID) references MatchResult (ResultID)
        on delete cascade,
    foreign key (TeamID) references Team (TeamID)
        on delete cascade
);
```

Stores the outcome for each team in a match. Composite PK (ResultID, TeamID) ensures each team appears once per result. IsWinner uses CHECK to allow only 0 or 1 (1 means winner and 0 means loser). Foreign keys make sure the referenced match result and team exist.

### 5.1.18 Sponsor

```
create table Sponsor
(
    SponsorID     varchar(10),
    Name          varchar(50) not null,
    Email         varchar(100),
    primary key (SponsorID)
);
```

Stores information about the sponsors of teams. SponsorID is the PK. Name is NOT NULL. Email is optional.



### 5.1.19 Sponsorship

```
create table Sponsorship
(
  SponsorID      varchar(10),
  OccurrenceID   varchar(10),
  TournamentSeriesID varchar(10),
  TeamID         varchar(10),
  Amount         numeric(12,2) check (Amount >= 0),
  primary key (SponsorID, OccurrenceID, TournamentSeriesID, TeamID),
  foreign key (SponsorID) references Sponsor (SponsorID)
    on delete cascade,
  foreign key (OccurrenceID) references TournamentOccurrence (OccurrenceID)
    on delete cascade,
  foreign key (TournamentSeriesID) references TournamentSeries (TournamentSeriesID)
    on delete cascade,
  foreign key (TeamID) references Team (TeamID)
    on delete cascade
);
```

Connects sponsors to tournament occurrences and teams. Composite PK keeps each sponsorship record unique. All attributes are FK references, and CASCADE makes sure sponsorship entries don't remain without their parent entities.

### 5.1.20 Staff

```
create table Staff
(StaffID      varchar(10),
 Name         varchar(50) not null,
 Role         varchar(30),
 TeamID       varchar(10),
 primary key (StaffID),
 foreign key (TeamID) references Team (TeamID)
      on delete set null
);
```

Contains team staff members. StaffID is the PK. Name is mandatory. TeamID is a foreign key but SET NULL is used because staff can still exist even if the team is removed.

### 5.1.21 TrainingSession

```
create table TrainingSession
(TrainingID    varchar(10),
 Date          date not null,
 Type          varchar(30),
 Duration      numeric(3,0), -- minutes it took to complete training session
 StaffID       varchar(10) not null,
 primary key (TrainingID),
 foreign key (StaffID) references Staff (StaffID)
      on delete cascade
);
```

Represents training sessions led by a staff member. StaffID is NOT NULL and references Staff. Duration represents the minutes it took to complete a training session. CASCADE removes sessions if the corresponding staff is deleted.

### 5.1.22 PlayerAttendsTrainingSession

```
create table PlayerAttendsTrainingSession
  (PlayerID      varchar(10),
   TrainingID    varchar(10),
   primary key (PlayerID, TrainingID),
   foreign key (PlayerID) references Player (PlayerID)
     on delete cascade,
   foreign key (TrainingID) references TrainingSession (TrainingID)
     on delete cascade
  );
```

Tracks attendance of players in training sessions. Composite PK (PlayerID, TrainingID) avoids duplicates. Both attributes are foreign keys with CASCADE, since attendance depends on both the player and the session.

### 5.1.23 Feedback

```
create table Feedback
(FeedbackID      varchar(10),
Notes           varchar(500),
Rating          numeric(2,1) check (Rating >= 1 and Rating <= 5),
StaffID         varchar(10) not null,
PlayerID        varchar(10) not null,
TrainingID      varchar(10) not null,
primary key (FeedbackID),
foreign key (StaffID) references Staff (StaffID)
    on delete cascade,
foreign key (PlayerID) references Player (PlayerID)
    on delete cascade,
foreign key (TrainingID) references TrainingSession (TrainingID)
    on delete cascade
);
```

Stores feedback from staff members about players' performance in a training session. FeedbackID is the PK. Rating has a CHECK (1–5). All foreign keys are required and use CASCADE because feedback cannot exist without the related staff, player, or session.

*The full SQL code for these queries could be found in the “DDL.sql” file.*

### ➤ DML Queries:

After constructing the full database schema using the CREATE TABLE statements, we populated the database with a complete set of INSERT INTO commands. The purpose of this phase was to generate a large, realistic dataset that allows us to properly test the database.

We began by inserting data into parent tables first, such as TournamentOrganizer, VideoGame, TournamentSeries, Team, Player, and Sponsor, to ensure that all referenced keys existed before inserting dependent tuples. This order was essential for avoiding foreign key violations, since later tables, like TournamentOccurrence, OfficialMatch, TeamMatchParticipation, and PlayerMatchStats, rely on these parent entries.

For the main entities in the esports environment, Team, Player, OfficialMatch, PlayerMatchStats, MatchResult, and TeamResult, we inserted at least 20 tuples each, since these tables represent the core gameplay and competition structure. Supporting tables such as Scrimmage, PlayerTeamHistory, Sponsorship, TrainingSession, Staff, Feedback, and PlayerAttendsTrainingSession were populated with around 10–15 tuples each. This provided enough data to test multi-step joins, grouping operations, and constraint behavior.

The data itself is random, non-uniform and realistic. The final dataset includes a natural variety of real-sounding names, different player roles, multiple regions and countries, mixed training types, etc. Dates, prize pools, accuracy values, and sponsorship amounts were chosen to fall within ranges that comply with the CHECK constraints defined earlier.

When populating relationship tables such as TournamentParticipation, TeamMatchParticipation, WonBy, PlayerMatchStats, TeamResult, Sponsorship, and PlayerAttendsTrainingSession, we ensured that every tuple referenced existing valid rows in parent tables. For example, each TeamMatchParticipation entry correctly pairs a valid TeamID with a valid MatchID, every MatchResult belongs to an existing OfficialMatch, and

each match includes precisely two TeamResult rows to represent the winning and losing teams.

Finally, test data was added when needed to produce results for queries that gave empty results. Overall, the populated data simulates a small but realistic esports database.

*The full SQL code for these queries could be found in the “DML.sql” file.*

## 6. Basic Queries

In this section, we described the details of the basic SQL queries that we tested. For each query, we provide an explanation of the query and a screenshot of the result that was obtained from the DBMS.

### 6.1 Query Containing a Cartesian Product

```
SELECT MENA.TeamName AS MENA_Team, NA.TeamName AS NorthAmerica_Team
FROM Team MENA, Team NA
WHERE MENA.Region = 'MENA'
AND NA.Region = 'North America';
```

This query generates every possible matchup between MENA teams and North American teams. It uses a Cartesian product to list all cross-region pairings.

*Output:*

|  | MENA_Team        | NorthAmerica_Team |
|--|------------------|-------------------|
|  | Beirut Blaze     | New York Voltage  |
|  | Dubai Mirage     | New York Voltage  |
|  | Istanbul Phoenix | New York Voltage  |
|  | Cairo Falcons    | New York Voltage  |
|  | Cedar Storm      | New York Voltage  |
|  | Lebanon Lions    | New York Voltage  |
|  | Beirut Blaze     | Toronto Nebula    |
|  | Dubai Mirage     | Toronto Nebula    |
|  | Istanbul Phoenix | Toronto Nebula    |
|  | Cairo Falcons    | Toronto Nebula    |

## 6.2 Query Containing a Natural Join

```
SELECT OccurrenceID, TournamentSeriesID, Name AS SeriesName, GameTitle, StartDate, EndDate, PrizePool
FROM TournamentOccurrence NATURAL JOIN TournamentSeries;
```

This query shows all tournament occurrences along with their tournament series information. The natural join automatically matches both tables on TournamentSeriesID.

*Output:*

|   | OccurrenceID | TournamentSeriesID | SeriesName       | GameTitle             | StartDate  | EndDate    | PrizePool |
|---|--------------|--------------------|------------------|-----------------------|------------|------------|-----------|
| ► | OC01         | TS01               | Nova Masters     | Nova Masters 2024     | 2024-01-10 | 2024-01-12 | 50000.00  |
|   | OC02         | TS01               | Nova Masters     | Nova Masters 2024     | 2024-06-05 | 2024-06-07 | 60000.00  |
|   | OC03         | TS02               | Summit Clash     | Summit Clash 2024     | 2024-02-15 | 2024-02-18 | 75000.00  |
|   | OC04         | TS02               | Summit Clash     | Summit Clash 2024     | 2024-08-01 | 2024-08-03 | 80000.00  |
|   | OC05         | TS03               | Phoenix Showdown | Phoenix Showdown 2024 | 2024-03-20 | 2024-03-22 | 40000.00  |
|   | OC06         | TS03               | Phoenix Showdown | Phoenix Showdown 2024 | 2024-09-10 | 2024-09-12 | 45000.00  |

## 6.3 Query Containing a Theta Join using “USING” Keyword

```
SELECT TS.TournamentSeriesID, TS.Name AS SeriesName, VG.Name AS VideoGame,
       TOG.Name AS OrganizerName, TOG.ContactEmail
FROM TournamentSeries TS
JOIN TournamentOrganizer TOG USING (OrganizerID)
JOIN VideoGame VG USING (VideoGameID);
```

This retrieves tournament series, their organizers, and the video game being played. The USING clause simplifies the join by merging identical column names.

*Output:*

| TournamentSeriesID | SeriesName        | VideoGame         | OrganizerName           | ContactEmail                 |
|--------------------|-------------------|-------------------|-------------------------|------------------------------|
| TS01               | Nova Masters      | Valorant          | Nova Esports Events     | contact@novaevents.com       |
| TS02               | Summit Clash      | League of Legends | Summit Gaming League    | info@summitleague.gg         |
| TS03               | Phoenix Showdown  | Overwatch 2       | Phoenix Tournament Co.  | admin@phoenixtournaments.com |
| TS04               | Vertex Global Cup | Counter-Strike 2  | Vertex Esports Group    | support@vertexeg.com         |
| TS05               | Apex Arena Series | Rocket League     | Apex Competitive Series | hello@apexseries.com         |



## 6.4 Query Containing Theta Join using “ON” Keyword

```
SELECT OM.MatchID, OM.ScheduledDate, OM.StartTime, OM.Type AS MatchType,
       Home.TeamName AS HomeTeam, Away.TeamName AS AwayTeam
FROM OfficialMatch OM
JOIN TeamMatchParticipation TMP_H
     ON OM.MatchID = TMP_H.MatchID AND TMP_H.Side = 'Home'
JOIN Team Home ON TMP_H.TeamID = Home.TeamID
JOIN TeamMatchParticipation TMP_A
     ON OM.MatchID = TMP_A.MatchID AND TMP_A.Side = 'Away'
JOIN Team Away ON TMP_A.TeamID = Away.TeamID
ORDER BY OM.ScheduledDate, OM.StartTime;
```

This query lists each official match along with the home and away teams. It uses ON-conditions to differentiate sides and orders the results chronologically.

*Output:*

| MatchID | ScheduledDate | StartTime | MatchType | HomeTeam          | AwayTeam          |
|---------|---------------|-----------|-----------|-------------------|-------------------|
| M001    | 2024-01-10    | 16:00:00  | BestOf3   | Atlas Esports     | Lebanon Lions     |
| M002    | 2024-01-10    | 18:00:00  | BestOf3   | Cedar Storm       | North Star Gaming |
| M003    | 2024-01-11    | 16:00:00  | BestOf3   | Seoul Dragons     | Tokyo Thunder     |
| M004    | 2024-01-11    | 18:00:00  | BestOf3   | Rio Guardians     | Berlin Titans     |
| M005    | 2024-02-15    | 17:00:00  | BestOf5   | Atlas Esports     | Seoul Dragons     |
| M006    | 2024-02-16    | 17:00:00  | BestOf5   | Lebanon Lions     | Tokyo Thunder     |
| M007    | 2024-03-20    | 17:00:00  | BestOf5   | Cedar Storm       | Rio Guardians     |
| M008    | 2024-03-21    | 17:00:00  | BestOf5   | North Star Gaming | Berlin Titans     |
| M009    | 2024-04-25    | 19:00:00  | BestOf5   | Paris Eclipse     | Cairo Falcons     |

## 6.5 Query Containing a Self-Join

```
SELECT P1.GamerTag AS Player1, P2.GamerTag AS Player2, P1.Country
FROM Player P1
JOIN Player P2 ON P1.Country = P2.Country
AND P1.PlayerID < P2.PlayerID;
```

This finds pairs of players who come from the same country. The self-join ensures players are compared with each other without duplication.

*Output:*

| Player1   | Player2    | Country |
|-----------|------------|---------|
| ShadowFox | LunarWolf  | Lebanon |
| ShadowFox | EchoRush   | Lebanon |
| ShadowFox | SolarFlare | Lebanon |
| ShadowFox | SilentStep | Lebanon |
| ShadowFox | CrystalOrb | Lebanon |
| LunarWolf | EchoRush   | Lebanon |
| LunarWolf | SolarFlare | Lebanon |
| LunarWolf | SilentStep | Lebanon |
| LunarWolf | CrystalOrb | Lebanon |

## 6.6 Query Containing the “DISTINCT” Keyword

```
SELECT DISTINCT Country
FROM Player
WHERE Country IS NOT NULL;
```

This returns a clean list of all countries represented by players, removing duplicates using DISTINCT.

*Output:*

| Country |
|---------|
| Lebanon |
| Jordan  |
| France  |
| Brazil  |
| Morocco |
| USA     |
| UK      |
| Japan   |
| Germany |

## 6.7 Query Containing the “LIKE” Keyword

```
SELECT PlayerID, GamerTag, Role, Country
FROM Player
WHERE GamerTag LIKE '%Fox%';
```

This query searches for players whose gamertag contains the substring “Fox.” The LIKE operator lets us do this.

*Output:*

| PlayerID | GamerTag  | Role    | Country |
|----------|-----------|---------|---------|
| P001     | ShadowFox | Duelist | Lebanon |

## 6.8 Query Containing the “ORDER BY” Keyword

```
SELECT T.TeamName, TMP.MatchID, OM.ScheduledDate, OM.StartTime,  
       OM.Type AS MatchType, TS.Name AS SeriesName, TS.GameTitle  
FROM Team T  
JOIN TeamMatchParticipation TMP ON T.TeamID = TMP.TeamID  
JOIN OfficialMatch OM ON TMP.MatchID = OM.MatchID  
JOIN TournamentSeries TS ON OM.TournamentSeriesID = TS.TournamentSeriesID  
WHERE T.TeamName = 'Beirut Blaze'  
ORDER BY OM.ScheduledDate ASC, OM.StartTime ASC;
```

This query retrieves all official matches played by Beirut Blaze, sorted chronologically by date and time.

*Output:*

| TeamName     | MatchID | ScheduledDate | StartTime | MatchType | SeriesName   | GameTitle         |
|--------------|---------|---------------|-----------|-----------|--------------|-------------------|
| Beirut Blaze | M014    | 2024-06-06    | 18:00:00  | BestOf3   | Nova Masters | Nova Masters 2024 |
| Beirut Blaze | M015    | 2024-08-01    | 16:00:00  | BestOf3   | Summit Clash | Summit Clash 2024 |

## 6.9 Query Containing the “UNION” Keyword

```
SELECT DISTINCT TeamID  
FROM TournamentParticipation  
UNION  
SELECT DISTINCT TeamID  
FROM Sponsorship;
```

Combines two sets of teams: those who participated in tournaments and those who received sponsorships. UNION removes duplicates.

*Output:*

| TeamID |
|--------|
| T001   |
| T002   |
| T003   |
| T004   |
| T005   |
| T006   |
| T007   |
| T008   |
| T009   |

## 6.10 Query Containing the “INTERSECT” Keyword

```
SELECT TeamID
FROM TournamentParticipation
INTERSECT
SELECT TeamID
FROM Sponsorship;
```

```
SELECT DISTINCT TP.TeamID
FROM TournamentParticipation TP
WHERE TP.TeamID IN
(SELECT DISTINCT TeamID
FROM Sponsorship);
```

(MySQL does not support INTERSECT so we included the MYSQL Equivelant)

Returns only teams that appear in both participation and sponsorship tables. These are the most active or marketable teams.

Output:

| TeamID |
|--------|
| T001   |
| T002   |
| T004   |
| T005   |
| T007   |
| T008   |
| T009   |
| T010   |
| T013   |

## 6.11 Query Containing the “EXCEPT” Keyword

```
SELECT TeamID
FROM TournamentParticipation
EXCEPT
SELECT TeamID
FROM Sponsorship;
```

```
SELECT DISTINCT TP.TeamID
FROM TournamentParticipation TP
WHERE TP.TeamID NOT IN
(SELECT DISTINCT TeamID
FROM Sponsorship);
```

(MySQL does not support EXCEPT so we included the MYSQL Equivelant)

Finds teams that participated in tournaments but did not receive sponsorships. This shows competitive teams that lack funding.

Output:

| TeamID |
|--------|
| T003   |
| T006   |
| T011   |
| T012   |
| T015   |

## 6.12 Query Containing a General Aggregate Function

```
SELECT SUM(PrizePool) AS TotalPrizePool
FROM TournamentOccurence;
```

Computes the total combined prize pool of all tournament occurrences. This is a single-value aggregate calculation.

*Output:*

| TotalPrizePool |
|----------------|
| 600000.00      |

## 6.13 Query Containing a Grouping Aggregate Function

```
SELECT T.TeamName, COUNT(DISTINCT TMP.MatchID) AS MatchesPlayed
FROM TeamMatchParticipation TMP
JOIN Team T ON TMP.TeamID = T.TeamID
GROUP BY T.TeamName
ORDER BY MatchesPlayed DESC;
```

Counts how many official matches each team played. The GROUP BY groups teams, and the result ranks them by how many matches they have played.

*Output:*

| TeamName         | MatchesPlayed |
|------------------|---------------|
| Seoul Dragons    | 3             |
| Tokyo Thunder    | 3             |
| Beirut Blaze     | 2             |
| Cairo Falcons    | 2             |
| Istanbul Phoenix | 2             |
| Paris Eclipse    | 2             |
| Athens Legacy    | 1             |
| Dubai Mirage     | 1             |
| Madrid Orbit     | 1             |

## 6.14 Query Containing both “GROUP BY” and “HAVING”

```
SELECT T.TeamName, VG.Name AS GameTitle,  
       SUM(CASE WHEN TR.IsWinner = 1 THEN 1 ELSE 0 END) AS Wins,  
       SUM(CASE WHEN TR.IsWinner = 0 THEN 1 ELSE 0 END) AS Losses  
FROM TeamResult TR  
JOIN MatchResult MR ON TR.ResultID = MR.ResultID  
JOIN OfficialMatch OM ON MR.MatchID = OM.MatchID  
JOIN TournamentSeries TS ON OM.TournamentSeriesID = TS.TournamentSeriesID  
JOIN VideoGame VG ON TS.VideoGameID = VG.VideoGameID  
JOIN Team T ON TR.TeamID = T.TeamID  
GROUP BY T.TeamName, VG.Name  
HAVING SUM(CASE WHEN TR.IsWinner = 1 THEN 1 ELSE 0 END) >= 2  
ORDER BY VG.Name, Wins DESC, T.TeamName;
```

Generates win–loss statistics for each team grouped by game title, but only includes teams with at least two wins. HAVING filters after aggregation.

*Output:*

| TeamName          | GameTitle   | Wins | Losses |
|-------------------|-------------|------|--------|
| North Star Gaming | Overwatch 2 | 2    | 0      |

## 6.15 Current Lebanese players and their teams

```
SELECT P.GamerTag, P.FirstName, P.LastName, P.Country,  
       T.TeamName, PTH.StartDate  
FROM Player P  
JOIN PlayerTeamHistory PTH ON P.PlayerID = PTH.PlayerID  
JOIN Team T ON PTH.TeamID = T.TeamID  
WHERE P.Country = 'Lebanon'  
AND PTH.EndDate IS NULL  
ORDER BY T.TeamName ASC, P.GamerTag ASC;
```

This Query lists all active Lebanese players, the team they play for, and the date they joined, ordered by team name and gamer tag.

*Output:*

| GamerTag   | FirstName | LastName | Country | TeamName      | StartDate  |
|------------|-----------|----------|---------|---------------|------------|
| LunarWolf  | Maya      | Haddad   | Lebanon | Beirut Blaze  | 2023-01-01 |
| ShadowFox  | Adam      | Khalil   | Lebanon | Beirut Blaze  | 2023-01-01 |
| EchoRush   | Sami      | Karim    | Lebanon | Cairo Falcons | 2023-02-02 |
| SolarFlare | Lara      | Nassar   | Lebanon | Cedar Storm   | 2023-08-01 |
| CrystalOrb | Sara      | Khatib   | Lebanon | Paris Eclipse | 2020-10-10 |
| SilentStep | Rami      | Saad     | Lebanon | Paris Eclipse | 2020-10-10 |



## 6.16 Staff members with high average feedback rating

```
SELECT S.StaffID, S.Name, S.Role,  
       AVG(F.Rating) AS AvgRating, COUNT(F.FeedbackID) AS NumFeedbacks  
FROM Staff S  
JOIN Feedback F ON S.StaffID = F.StaffID  
GROUP BY S.StaffID, S.Name, S.Role  
HAVING AVG(F.Rating) >= 4.0  
ORDER BY AvgRating DESC, NumFeedbacks DESC;
```

Shows staff members who have an average feedback rating of at least 4.0, along with how many feedback entries they received. This helps show high-performing staff based on player feedback, which could be used for promotions, bonuses, or assigning them to top teams.

*Output:*

| StaffID | Name             | Role              | AvgRating | NumFeedbacks |
|---------|------------------|-------------------|-----------|--------------|
| ST10    | Sophia Wilson    | Team Manager      | 4.90000   | 1            |
| ST03    | James Miller     | Assistant Coach   | 4.80000   | 1            |
| ST05    | Lucas Pereira    | Performance Coach | 4.70000   | 1            |
| ST07    | Daniel Green     | Head Coach        | 4.60000   | 1            |
| ST02    | Nour Abou Hassan | Analyst           | 4.10000   | 2            |
| ST01    | Karim Youssef    | Head Coach        | 4.00000   | 2            |

## 6.17 Which teams trained regularly

```
SELECT T.TeamName,  
       COUNT(TS.TrainingID) AS NumTrainingSessions  
FROM TrainingSession TS  
JOIN Staff S ON TS.StaffID = S.StaffID  
JOIN Team T ON S.TeamID = T.TeamID  
GROUP BY T.TeamName  
HAVING COUNT(TS.TrainingID) >= 3  
ORDER BY NumTrainingSessions DESC;
```

This counts how many training sessions were conducted per team and lists teams that had more than 2 training sessions.

*Output:*

| TeamName      | NumTrainingSessions |
|---------------|---------------------|
| Atlas Esports | 5                   |
| Lebanon Lions | 4                   |

## 7. Advanced Queries

In this section, we present a set of advanced SQL queries that were tested on the database. These queries use more advanced SQL operations and involve more complex logic, such as multi-table joins, nested subqueries, and aggregate functions with grouping and conditions.

For each advanced query, we explain its purpose, describe how it operates on the database schema, and include a screenshot of the result obtained from the DBMS.

### 7.1 A query that contains a set membership nested query

We use this query to retrieve all players who participated in a specific tournament series (Summit Clash). We do this by using a nested query with the IN operator to check whether a player appears in the set of players who played matches belonging to that series.

```
SELECT DISTINCT P.GamerTag, P.PlayerID
FROM Player P
WHERE P.PlayerID IN
    (SELECT DISTINCT PMS.PlayerID
     FROM PlayerMatchStats PMS
     JOIN OfficialMatch OM ON PMS.MatchID = OM.MatchID
     JOIN TournamentSeries TS ON OM.TournamentSeriesID = TS.TournamentSeriesID
     WHERE TS.Name = 'Summit Clash');
```

*Output:*

| GamerTag  | PlayerID |
|-----------|----------|
| ShadowFox | P001     |
| IronHeart | P003     |
| ZenBlade  | P009     |
| Skyline   | P010     |
| NightOwl  | P011     |
| EchoRush  | P012     |

## 7.2 A query that contains a set comparison nested query

We use this query to identify the team or teams with the highest number of wins across all matches. We do this by using a set comparison nested query with the ALL keyword to compare each team's win count against every other team.

```
SELECT T.TeamName
FROM Team T
JOIN TeamResult TR ON T.TeamID = TR.TeamID
GROUP BY T.TeamName
HAVING SUM(CASE WHEN TR.IsWinner = 1 THEN 1 ELSE 0 END) >= ALL
(SELECT SUM(CASE WHEN TR2.IsWinner = 1 THEN 1 ELSE 0 END)
FROM TeamResult TR2
JOIN Team T2 ON TR2.TeamID = T2.TeamID
GROUP BY T2.TeamName);
```

Output:

| TeamName      |
|---------------|
| Atlas Esports |
| Cedar Storm   |

## 7.3 A query that contains a set cardinality nested query

We use this query to find teams that have never won an official match. We do this by using a NOT EXISTS nested query that checks whether any winning record exists for a given team.

```
SELECT T.TeamName
FROM Team T
WHERE NOT EXISTS
(SELECT *
FROM TeamResult TR
WHERE TR.TeamID = T.TeamID
AND TR.IsWinner = 1);
```

Output:

| TeamName         |
|------------------|
| Rio Guardians    |
| Madrid Orbit     |
| New York Voltage |
| Sydney Wave      |
| Warsaw Reapers   |
| Athens Legacy    |

## 7.4 A query that contains two or more nesting levels

We use this query to retrieve players who attended training sessions led by staff members from teams that participated in a specific tournament series (Summit Clash). We do this by using multiple levels of nested queries that link players to training sessions, staff, teams, and tournament participation.

```
SELECT DISTINCT P.GamerTag
FROM Player P
WHERE P.PlayerID IN
    (SELECT PATS.PlayerID
     FROM PlayerAttendsTrainingSession PATS
     WHERE PATS.TrainingID IN
         (SELECT TSN.TrainingID
          FROM TrainingSession TSN
          WHERE TSN.StaffID IN
              (SELECT S.StaffID
               FROM Staff S
               WHERE S.TeamID IN
                   (SELECT TP.TeamID
                    FROM TournamentParticipation TP
                    JOIN TournamentSeries TS
                      ON TP.TournamentSeriesID = TS.TournamentSeriesID
                    WHERE TS.Name = 'Summit Clash'))));
```

*Output:*

| GamerTag   |
|------------|
| CrimsonAce |
| StormRider |
| VoltStrike |
| FrostNova  |

## 7.5 A query that contains the SQL division operation

We use this query to find teams that participated in all occurrences of a specific tournament series. We do this by implementing the SQL division operation using NOT EXISTS and EXCEPT.

```
SELECT T.TeamName
FROM Team T
WHERE NOT EXISTS
    ( (SELECT O.OccurrenceID
      FROM TournamentOccurrence O
      WHERE O.TournamentSeriesID = 'TS001')
    EXCEPT
    (SELECT TP.OccurrenceID
     FROM TournamentParticipation TP
     WHERE TP.TeamID = T.TeamID
       AND TP.TournamentSeriesID = 'TS001')
    );
```

*(MySQL does not support EXCEPT so we included the MYSQL Equivalent)*

```
SELECT T.TeamName
FROM Team T
WHERE NOT EXISTS
    (SELECT O.OccurrenceID
     FROM TournamentOccurrence O
     WHERE O.TournamentSeriesID = 'TS001'
     AND O.OccurrenceID NOT IN
        (SELECT TP.OccurrenceID
         FROM TournamentParticipation TP
         WHERE TP.TeamID = T.TeamID
           AND TP.TournamentSeriesID = 'TS001'));
```

Output:

| TeamName          |
|-------------------|
| Atlas Esports     |
| Lebanon Lions     |
| Cedar Storm       |
| North Star Gaming |
| Seoul Dragons     |
| Tokyo Thunder     |
| Rio Guardians     |

## 7.6 A query that contains a nested query in the FROM statement

We use this query to generate a leaderboard of players based on their total kills and deaths. We do this by using a nested query in the FROM clause to precompute aggregated statistics before joining them with the Player table.

```
SELECT P.GamerTag, Stats.TotalKills, Stats.TotalDeaths
FROM Player P
JOIN (
    SELECT PlayerID,
           SUM(Kills) AS TotalKills,
           SUM(Deaths) AS TotalDeaths
    FROM PlayerMatchStats
    GROUP BY PlayerID
) AS Stats
ON P.PlayerID = Stats.PlayerID
ORDER BY Stats.TotalKills DESC;
```

*Output:*

| GamerTag   | TotalKills | TotalDeaths |
|------------|------------|-------------|
| ShadowFox  | 85         | 38          |
| IronHeart  | 76         | 36          |
| LunarWolf  | 55         | 32          |
| VoltStrike | 25         | 11          |
| SolarFlare | 23         | 12          |
| RapidFire  | 21         | 10          |
| PixelMage  | 20         | 7           |

## 7.7 A query that contains a nested query in the SELECT statement

We use this query to show the number of currently active players per team. We do this by using a scalar nested query inside the SELECT statement that counts ongoing player memberships for each team.

```
SELECT T.TeamName,  
       (SELECT COUNT(*)  
        FROM PlayerTeamHistory PTH  
        WHERE PTH.TeamID = T.TeamID  
        AND PTH.EndDate IS NULL) AS CurrentPlayers  
FROM Team T  
ORDER BY CurrentPlayers DESC, T.TeamName;
```

*Output:*

| TeamName          | CurrentPlayers |
|-------------------|----------------|
| Atlas Esports     | 3              |
| Beirut Blaze      | 2              |
| Cairo Falcons     | 2              |
| Cedar Storm       | 2              |
| North Star Gaming | 2              |
| Paris Eclipse     | 2              |



## 7.8 An UPDATE query that contains a “CASE” statement

We use this query to automatically update each player's status based on how many matches they have played. We do this by using an UPDATE statement with a CASE expression and nested queries to categorize players as New, Active, or Core.

```
UPDATE Player P
  SET Status = CASE
    WHEN (SELECT COUNT(*)
          FROM PlayerMatchStats PMS
          WHERE PMS.PlayerID = P.PlayerID) >= 10
    THEN 'Core'
    WHEN (SELECT COUNT(*)
          FROM PlayerMatchStats PMS
          WHERE PMS.PlayerID = P.PlayerID) BETWEEN 1 AND 9
    THEN 'Active'
    ELSE 'New'
  END;
```

## 7.9 A query that contains an OUTER JOIN

We use this query to list all teams along with their sponsorship information, including teams without sponsors. We do this by using LEFT OUTER JOIN operations to preserve teams that do not appear in the Sponsorship table.

```
SELECT T.TeamName,
       S.SponsorID,
       S.Name AS SponsorName,
       SP.Amount
FROM Team T
LEFT OUTER JOIN Sponsorship SP ON T.TeamID = SP.TeamID
LEFT OUTER JOIN Sponsor S     ON SP.SponsorID = S.SponsorID
ORDER BY T.TeamName ASC;
```

Output:

| TeamName         | SponsorID | SponsorName        | Amount   |
|------------------|-----------|--------------------|----------|
| Berlin Titans    | SP05      | Aurora Headsets    | 9000.00  |
| Cairo Falcons    | SP07      | Nova Chairs        | 19000.00 |
| Cedar Storm      | SP02      | Pulse Energy Drink | 12500.00 |
| Dubai Mirage     | SP08      | Starlight Snacks   | 8000.00  |
| Istanbul Phoenix | NULL      | NULL               | NULL     |

## 7.10 A “CREATE ASSERTION” query that contains a condition on at least two database tables

We use this query to enforce a global integrity constraint on the total number of players and staff members in the system. We do this by defining a CREATE ASSERTION in standard SQL and simulating it in MySQL using a trigger that checks multiple tables before insertion.

```
CREATE ASSERTION TotalPeopleLimit
CHECK (
    (SELECT COUNT(*) FROM Player) +
    (SELECT COUNT(*) FROM Staff) <= 200
);
```

```
DELIMITER //
CREATE TRIGGER check_total_people_before_insert
BEFORE INSERT ON Player
FOR EACH ROW
BEGIN
    DECLARE total INT;
    SELECT (SELECT COUNT(*) FROM Player)
        + (SELECT COUNT(*) FROM Staff)
    INTO total;
    IF total >= 200 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Total people cant exceed 200';
    END IF;
END//
DELIMITER ;
```

*Proof:*

In standard SQL, this constraint is expressed using a CREATE ASSERTION statement, which ensures that no database state exists where the violation condition holds. Since MySQL does not support assertions, we implemented an equivalent mechanism using triggers.

We use the trigger to enforce the same rule incrementally on every modification. We do this by evaluating the same EXISTS / NOT EXISTS condition inside the trigger body and raising an error with SIGNAL whenever the constraint would be violated.

To prove the equivalence, we tested both valid and invalid cases: valid inserts are accepted without errors, while invalid inserts cause the trigger to fire and the DBMS to reject the operation.

## 7.11 A “CREATE VIEW” query

We use this query to create a reusable summary of each team’s win–loss record. We do this by defining a database view that aggregates match results and presents them as a virtual table.

```
CREATE VIEW TeamWinLoss AS
  SELECT T.TeamID,
         T.TeamName,
         SUM(CASE WHEN TR.IsWinner = 1 THEN 1 ELSE 0 END) AS Wins,
         SUM(CASE WHEN TR.IsWinner = 0 THEN 1 ELSE 0 END) AS Losses
  FROM Team T
  LEFT JOIN TeamResult TR ON T.TeamID = TR.TeamID
  GROUP BY T.TeamID, T.TeamName;
-- Testing the View table:
SELECT * FROM TeamWinLoss;
```

(We tested the View Table by running the “*SELECT \*FROM TeamWinLoss;*” line)

Output:

| TeamID | TeamName          | Wins | Losses |
|--------|-------------------|------|--------|
| T001   | Atlas Esports     | 4    | 1      |
| T002   | Lebanon Lions     | 1    | 3      |
| T003   | Cedar Storm       | 4    | 1      |
| T004   | North Star Gaming | 2    | 2      |
| T005   | Seoul Dragons     | 2    | 2      |

## 7.12 A Trigger that is created using the “CREATE TRIGGER” statement

We use this trigger to automatically mark players as active when they start participating in matches. We do this by using an AFTER INSERT trigger that updates the Player table whenever new match statistics are recorded.

```
DELIMITER //
CREATE TRIGGER mark_player_active_after_stats
AFTER INSERT ON PlayerMatchStats
FOR EACH ROW
BEGIN
    UPDATE Player
    SET Status = 'Active'
    WHERE PlayerID = NEW.PlayerID
    AND (Status IS NULL OR Status = 'New');
END//
DELIMITER ;
```

### *Testing:*

We first reset player status so that the trigger has something to change:

```
-- Testing
UPDATE Player
SET Status = 'New'
WHERE PlayerID = 'P017';

SELECT PlayerID, GamerTag, Status
FROM Player
WHERE PlayerID = 'P017';
```

This was the output before the trigger was activated:

| PlayerID | GamerTag   | Status |
|----------|------------|--------|
| P017     | SilentStep | New    |

Now we insert a NEW match stat which activates the trigger:

```
INSERT INTO PlayerMatchStats
(PlayerID, MatchID, Kills, Assists, Deaths, DamageDealt, Accuracy, Healing, Objective)
VALUES ('P017', 'M015', 12, 6, 7, 3100, 48.20, 200, 'Trigger testing insert');

SELECT PlayerID, GamerTag, Status
FROM Player
WHERE PlayerID = 'P017';
```

This was the output after the trigger was activated:

| PlayerID | GamerTag   | Status |
|----------|------------|--------|
| P017     | SilentStep | Active |

### 7.13 An advanced function or stored procedure

We use this function to compute the overall KDA ratio of any player. We do this by writing a stored function that aggregates kills, assists, and deaths and safely calculates the KDA value.

```
DELIMITER //
CREATE FUNCTION PlayerKDA(p_player_id VARCHAR(10))
RETURNS DECIMAL(6,2)
> BEGIN
    DECLARE k INT DEFAULT 0;
    DECLARE a INT DEFAULT 0;
    DECLARE d INT DEFAULT 0;
    DECLARE kda DECIMAL(6,2);
    SELECT SUM(Kills)
    INTO k
    FROM PlayerMatchStats
    WHERE PlayerID = p_player_id;
    SELECT SUM(Assists)
    INTO a
    FROM PlayerMatchStats
    WHERE PlayerID = p_player_id;
    SELECT SUM(Deaths)
    INTO d
    FROM PlayerMatchStats
    WHERE PlayerID = p_player_id;
    IF k IS NULL THEN SET k = 0; END IF;
    IF a IS NULL THEN SET a = 0; END IF;
    IF d IS NULL THEN SET d = 0; END IF;
    IF d = 0 THEN
        SET kda = k + a;
    ELSE
        SET kda = (k + a) / d;
    END IF;
    RETURN kda;
END //
DELIMITER ;
```

*Calling the function:*

```
-- running the function
SELECT PlayerKDA('P001') AS KDA;
```

This will calculate the KDA of Player with the ID 'P001' and then return the KDA as shown:

| KDA  |
|------|
| 2.79 |

## 7.14 Retrieve the KDA ratio of a player for the past three months

```
SELECT
    p.PlayerID, p.GamerTag,
    CASE
        WHEN SUM(pms.Deaths) = 0 THEN
            SUM(pms.Kills) + SUM(pms.Assists)
        ELSE
            (SUM(pms.Kills) + SUM(pms.Assists)) / SUM(pms.Deaths)
        END AS KDA_Last_3_Months
FROM Player p
JOIN PlayerMatchStats pms
    ON p.PlayerID = pms.PlayerID
JOIN OfficialMatch m
    ON pms.MatchID = m.MatchID
WHERE p.PlayerID = 'P001'
    AND m.ScheduledDate >= DATE_SUB(CURDATE(), INTERVAL 3 MONTH)
    AND m.ScheduledDate <= CURDATE()
GROUP BY
    p.PlayerID,
    p.GamerTag;
```

The function `PlayerKDA_Last3Months` computes the KDA ratio for a specific player over the past three months. It aggregates the total number of kills, assists, and deaths recorded in the `PlayerMatchStats` table for the given player and considers only matches whose scheduled dates fall within the last three months.

*Output:*

| PlayerID | GamerTag  | KDA_Last_3_Months |
|----------|-----------|-------------------|
| P001     | ShadowFox | 2.8214            |

## 8. Conclusion

This project resulted in the successful design and implementation of a complete relational database for an Esports Team Performance Tracker. Through a two-phase approach, we were able to model a real-world esports performance tracker database system. The final implementation supports tracking teams, players, matches, tournaments, training sessions, staff involvement, and performance statistics in an integrated manner.

In Phase 1, the database requirements, ER model, ER breakdown, and relational model provided a clear and accurate representation of the system's data and business rules. This phase ensured that complex relationships such as player-team history, tournament occurrences, match participation, and sponsorships were logically and correctly modeled before implementation.

In Phase 2, the database was implemented in MySQL using DDL and DML statements. We used constraints to help maintain data integrity, while the populated dataset allowed for meaningful testing. Basic and advanced queries demonstrated that the database can successfully support analytical tasks such as evaluating player performance, generating team statistics, tracking training activity, and reviewing tournament outcomes. The use of views, triggers, and stored functions further enhanced the database by automating logic and enabling reusable performance calculations, such as KDA and status updates.

While the current implementation meets the project requirements, several improvements could further enhance the system. Stored procedures could be expanded to generate automated reports, such as monthly team summaries or player progression reports. Additionally, enforcing more complex constraints through additional triggers and using transaction control for bulk inserts would improve reliability. Future extensions could also include role-based access control or a front-end interface to make the system more usable for non-technical users.

Overall, this project demonstrates how proper database design combined with effective implementation techniques can support data-driven decision-making in esports organizations. The resulting database provides a solid foundation that can be optimized and expanded as the needs of the system grow.